Assignment 1: PyTorch Foundations
Image and Text Classification

Generative AI & LLM, Fall 2026
Umair Ali (MSDS24066)

Submission links

- GitHub repository (private, TA added as collaborator): https://github.com/Umair-Ali-804/LLM_Generative_AI_Assignment01
- Google Drive model checkpoints: https://drive.google.com/drive/folders/1PsVnznjIOs5PLRfHBxuf5OWr_HffjEzt
- Weights & Biases, image runs: https://forge.coreweave.com/wandb/aliumair64488-information-technology-university/img_classifier
- Weights & Biases, text runs: https://forge.coreweave.com/wandb/aliumair64488-information-technology-university/txt_classifier
- AI conversation link (ChatGPT): https://chatgpt.com/c/6ad4cf9e-9d38-83e8-8605-35908d17b685

1 Background Reading

1.1

(2 points) How does PyTorch's autograd system compute gradients, and why is this feature essential for training neural networks?

Answer. During the forward pass PyTorch records every differentiable operation on tensors with `requires_grad=True` in a dynamic computation graph; each result stores the function that produced it (`grad_fn`). Calling `loss.backward()` walks this graph from the loss back to the leaves (reverse-mode automatic differentiation), applies the chain rule at each node and accumulates $\partial \text{loss}/\partial\theta$ in each parameter's `.grad`. This is essential because training is gradient descent over thousands to billions of parameters: deriving those gradients by hand is impractical and error-prone, while autograd gives exact gradients for any differentiable program, including Python control flow. The optimizer then reads `.grad` to update the weights and `zero_grad()` clears the accumulated values.

1.2

(2 points) Why is it beneficial to use something like wandb.log() over just printing the loss or accuracy every epoch?

Answer. Printed numbers disappear with the terminal and cannot be compared. `wandb.log()` stores metrics persistently indexed by step or epoch, draws live plots, and saves hyperparameters, code state and system metrics with each run. Runs can be overlaid and filtered in tables (e.g. SGD vs. Adam), shared by link, reproduced later, and extended with images or histograms; a diverging run can be noticed and stopped remotely.

1.3

(2 points) What do the TruncateToMaxLen and PadToMaxLen classes do in txt_classifier.py? Explain when and why each one is used.

Answer. Both turn variable-length token-id lists into fixed-length ones so a batch can be stacked into one `(batch, max_len)` tensor. `TruncateToMaxLen` cuts an article longer than `max_len` to its first `max_len` tokens, bounding memory and compute so a few very long articles do not dominate. `PadToMaxLen` appends `CorpusInfo.pad_idx` to shorter articles until they have exactly `max_len` tokens. Truncation runs first, padding second, and both are used only when `--max_len` is positive; with `-1` neither runs, which only works with batch size 1. The padding index is given to the embedding layer as `padding_idx`, so padded positions have a zero, non-updated embedding and are excluded from the mean in `EmbeddingBag`.

2 Image Classification (ungraded)

All image runs: SGD, learning rate 0.001, batch size 8, 5 epochs, seed 42.

2.1

(3 points) Run the image classifier with the default hyperparameters. Using wandb, plot the training loss of each batch vs. the total number of examples seen so far by the optimizer (average loss within each batch). Name your run "neural-the-narwhal" and include the legend in your graph.

Answer. Run `neural-the-narwhal` (MLP, SGD, lr 0.001, batch size 8, 5 epochs). The per-batch loss is noisy but trends down from about 1.1 to about 0.1–0.3 over the 5 epochs, with an occasional spike (e.g. about 1.8 near 2000 examples) caused by a hard batch of 8 images.

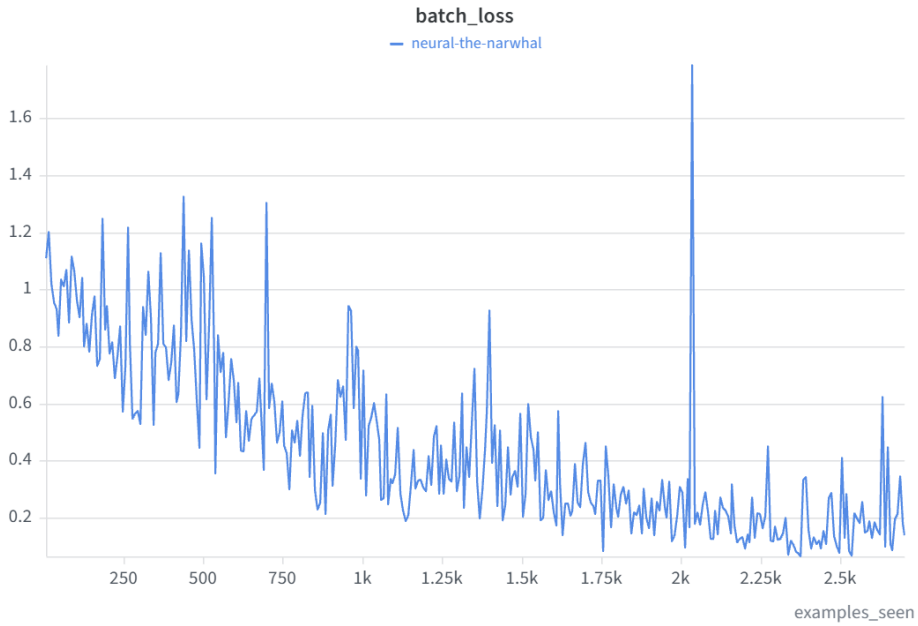


Figure 1: Per-batch training loss vs. examples seen (wandb, run neural-the-narwhal).

2.2

(4 points) Using wandb, plot the train loss and the validation loss on the same plot with the number of epochs on the horizontal axis (each averaged over the entire dataset).

Answer. Training loss falls steadily (0.587 to 0.129) while validation loss flattens at about 0.66–0.68 after epoch 2, so the gap widens: the model is overfitting.

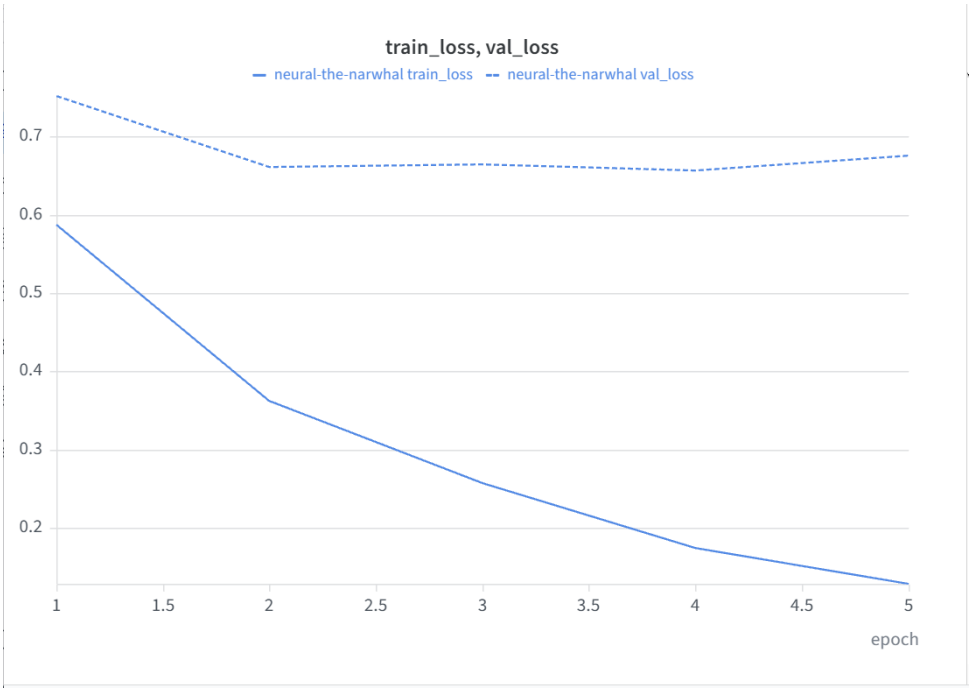


Figure 2: Train vs. validation loss per epoch (wandb, run neural-the-narwhal).

2.3

(4 points) Using wandb, plot the train accuracy and validation accuracy on the same plot with the number of epochs on the horizontal axis.

Answer. Training accuracy reaches 99.6% by epoch 5, while validation accuracy peaks at 73.3% (epoch 2) and ends at 69.0%. The MLP has 100.9M parameters for a few hundred training images.

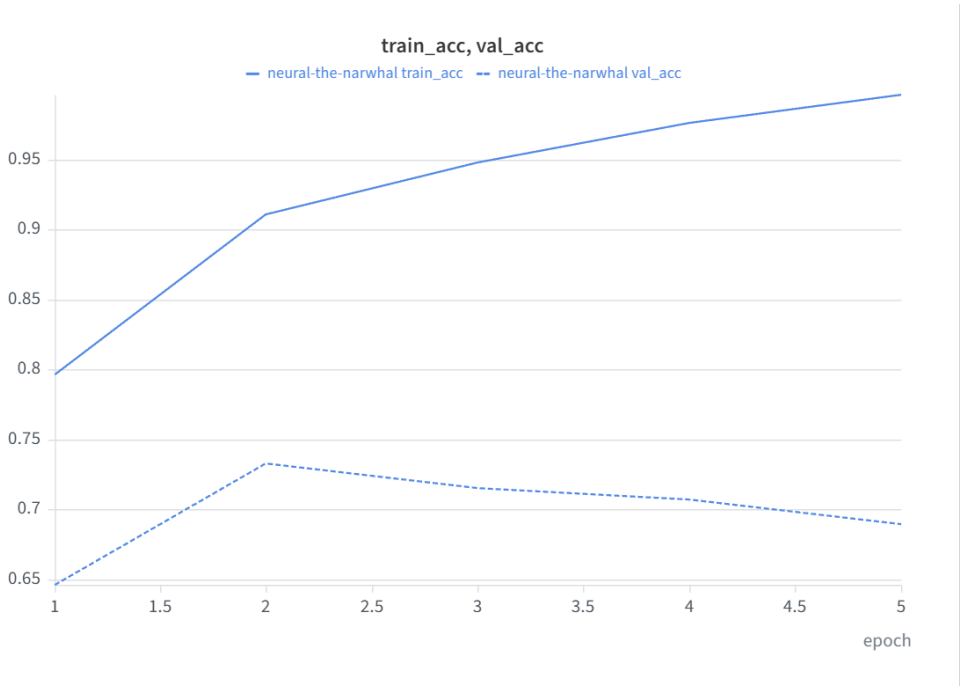


Figure 3: Train vs. validation accuracy per epoch (wandb, run neural-the-narwhal).

2.4.a

(3 points) What is the test accuracy with color images vs. grayscale images?

Answer.

Dataset	Image type	Accuracy
Test	Color	61.7%
Test	Grayscale	56.5%

2.4.b

(2 points) Does this support the hypothesis that the classifier wasn't really learning anything besides the color of the animals? Provide justification for your answer.

Answer. Grayscale lowers test accuracy by about 5 points, so colour helps, but accuracy stays well above chance (about 33%), so the classifier is not using colour alone: brightness patterns, texture and coarse shape also carry signal. With one seed and a test set of about 115 images, a 5-point gap is within plausible noise.

2.5.a

(2 points) Show the first batch of the test dataset captioned appropriately on wandb.

2.5.b

(2 points) Pick a few of the images on which the model made an error, and try to explain why it might have had difficulty with those images.

Answer. I did not complete this part (the captioned test-batch images and error analysis); it is an ungraded task.

2.6

Define a new model implementing the given computation graph (Conv k=4 → LayerNorm → Conv k=7 → LayerNorm → Conv k=1 → GELU → Conv k=1 → AvgPool k=2 → Flatten → Linear).

2.6.a

(1 point) Report the final train/test accuracy of your original model and your new model.

Answer.

Dataset	Accuracy (Original Model)	Accuracy (New Model)
Train	99.6%	89.6%
Test	61.7%	59.1%

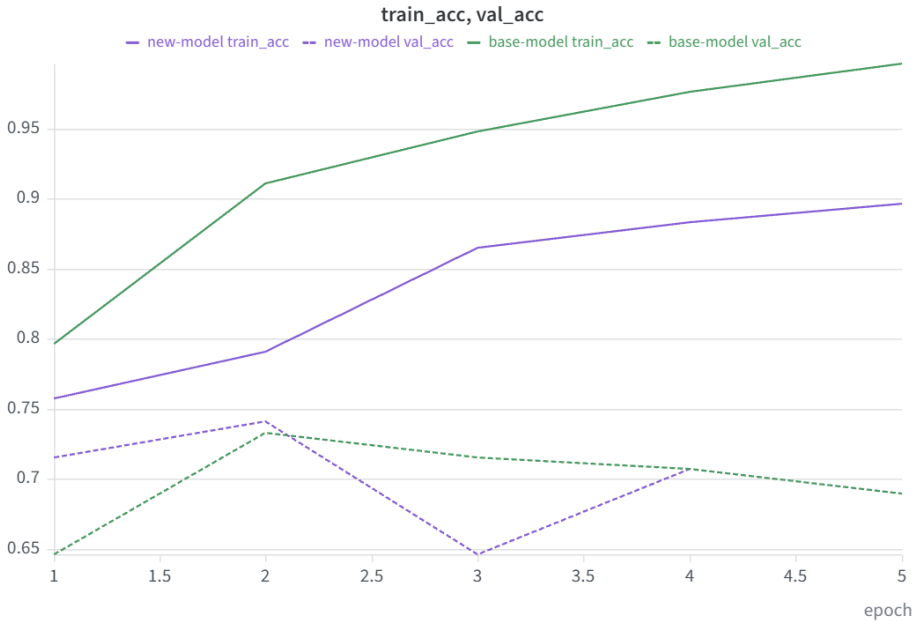


Figure 4: Train/validation accuracy, base-model vs. new-model (wandb).

2.6.b

(4 points) With wandb, plot the train accuracy and validation accuracy on the same plot with the number of epochs on the horizontal axis. Include a run with the original model and a run with your new model. Name your runs "new-model" and "base-model" and include a legend.

Answer. See the figure above: the CNN's training accuracy rises more slowly (89.6% vs. 99.6%) and both reach about 69% validation accuracy.

2.6.c

(2 points) How many parameters were in your original model and new model?

Answer. Original model: 100,928,003 parameters. New model: 1,268,867 parameters.

2.6.d

(2 points) Did your new model perform better or worse on the test set than the original model? What about the training set? What factors might explain the differences in accuracy you observed between the models?

Answer. The CNN has about 80× fewer parameters and slightly lower train and test accuracy (59.1% vs. 61.7% test). Plausible reasons: weight sharing gives a better inductive bias but both models got only 5 epochs of plain SGD at lr 0.001, so the CNN is probably under-trained; differences of a few points on about 115 test images are also within noise.

Learning-rate effect (MLP, SGD).

lr	final train acc	final val acc	final val loss
0.0001	83.7%	72.4%	0.738
0.001	99.6%	69.0%	0.676
0.01	99.1%	73.3%	0.936

Larger learning rates fit the training set faster (99.1% train accuracy at 0.01) and raise validation loss (0.936), i.e. overfit sooner; the smallest rate trains more slowly.

3 Text Classification (graded)

3.1

(3 points) Generate a histogram of the lengths of the raw news articles without truncation or padding.

Answer. Histogram of raw article lengths (training split, tokens, no truncation or padding).

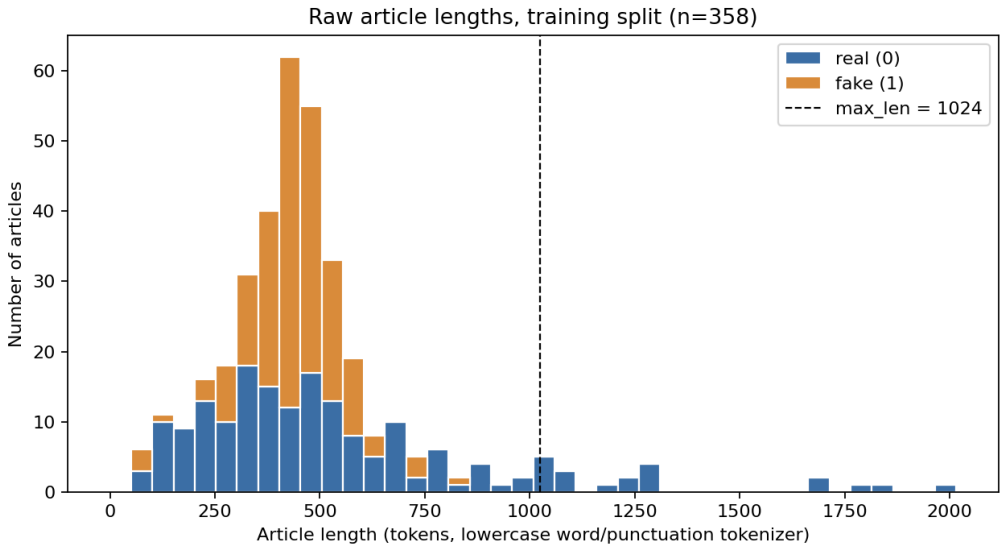


Figure 5: Raw article length histogram.

The 358 training articles have a median of 441 tokens, mean 483, 95th percentile 1034 and maximum 2016; 5.3% exceed 1024 tokens. Real articles average 530 tokens and generated ones 436. The distribution is right-skewed, so a cap of 1024 keeps almost all articles intact while bounding memory.

3.2.a

(1 point) Perform the three runs (batch size 1 / max_len −1; batch size 1 / max_len 1024; batch size 32 / max_len 1024) and compare the runtime of each run.

Answer. Runtime (SGD, lr 5, 5 epochs, GPU, same seed; includes per-epoch evaluation).

Batch Size	Max Length	Runtime (seconds)
1	None	5.32
1	1024	5.84
32	1024	1.88

Batch size 32 with padding to 1024 is 2.8× faster than batch size 1: it makes 1/32 as many optimizer steps, each a vectorised pass. At batch size 1, padding/truncation to 1024 makes almost no difference (5.3 s vs. 5.8 s), since most articles are already shorter than 1024 and the model is tiny.

3.2.b

(1 point) Now do one more run (batch size 32, max_len −1) and report the error that you get.

Answer. Error for `--batch_size 32 --max_len -1`:

```
File ".../txt_classifier.py", line 119, in __call__
    text = torch.stack([torch.tensor(x, dtype=torch.long) for x, _ in batch])
RuntimeError: stack expects each tensor to be equal size,
              but got [140] at entry 0 and [517] at entry 1
```

3.2.c

(2 points) Why does this last run yield this error?

Answer. The collate function (`PadSequence`) builds a batch with `torch.stack`, which needs all tensors to have the same shape. Without truncation/padding the articles have different token counts (here 140 vs. 517), so 32 of them cannot be stacked into one `(32, L)` tensor. At batch size 1 nothing needs stacking, so that case works.

3.3

(2 points) Run the code with the existing SGD optimizer and learning rate (5). Then run again with torch.optim.Adam at its default learning rate (0.001). For both runs, report the validation and test accuracies.

Answer. SGD vs. Adam (bag-of-embeddings model, batch size 1, no truncation, 5 epochs).

	Val. Acc.	Test Acc.
SGD (lr 5)	94.7%	93.2%
Adam (lr 0.001)	97.4%	98.6%

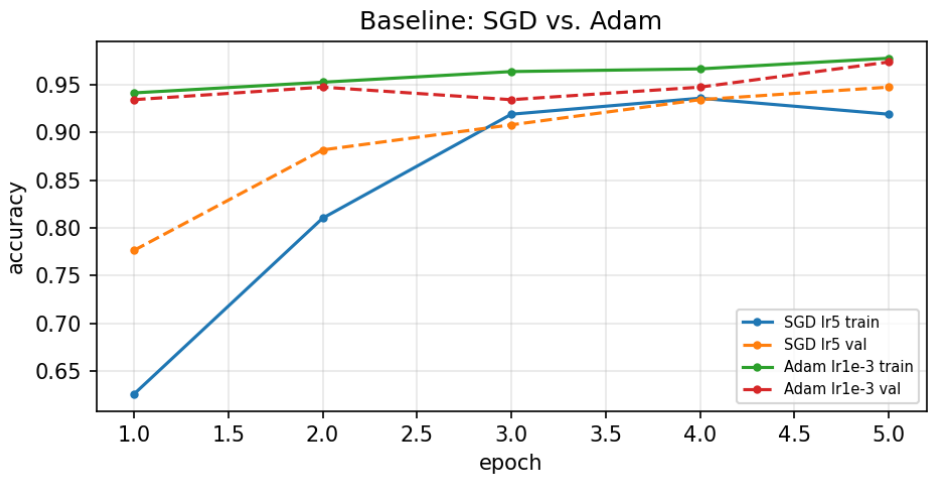


Figure 6: Baseline: SGD vs. Adam, train/validation accuracy.

Adam reaches higher accuracy and is already at 93% validation accuracy after one epoch, while SGD needs the large rate of 5 (with gradient clipping) to move the embeddings. Adam rescales each parameter's step by running estimates of the gradient's moments, so rarely seen words still get useful updates. Adam is used for all later experiments.

Learning-rate sweep (baseline model; compared on validation accuracy only, test accuracy never used for selection).

Optimizer	Learning rate	Final train acc	Final val acc
SGD	0.5	79.3%	85.5%
SGD	5.0	91.9%	94.7%
SGD	50.0	98.3%	93.4%
ADAM	0.0001	93.0%	81.6%
ADAM	0.001	97.8%	97.4%
ADAM	0.01	100.0%	97.4%

SGD is very sensitive: lr 0.5 under-trains (85.5% val) and lr 50 works but is noisy; Adam at 1e-4 is too slow in 5 epochs, while 1e-3 and 1e-2 reach 97.4% validation accuracy. Differences of one or two validation examples (76 examples) are within noise.

3.4

Replace the simple TextClassificationModel with an LSTM-based classifier using nn.Embedding (not EmbeddingBag), nn.LSTM with batch_first=True, a pooling layer after the LSTM outputs, the Adam optimizer, and the padding tokens defined in CorpusInfo.

Answer. Implemented as described in 3.4.a below (BiLSTM, `nn.Embedding`, masked max-pooling, Adam, CorpusInfo padding index).

3.4.a

(2 points) In math, pseudocode, or a computation graph, describe the structure of your new model.

Answer. Input tokens $x_1..x_T$ right-padded with PAD, true length n:

$$e_t = E[x_t] \quad (\texttt{nn.Embedding}, \text{padding_idx} = \text{PAD}, d = 64)$$
$$(h_1, \ldots, h_n) = \text{BiLSTM}(e_1, \ldots, e_n) \quad \text{(hidden 64 per direction, batch_first=True, packed)}, \; h_t \in \mathbb{R}^{128}$$
$$p = \max_{t \le n} h_t \quad \text{(max-pool over real timesteps; padded steps masked)}$$
$$\text{logits} = W \, \text{dropout}(p, 0.3) + b, \; W \in \mathbb{R}^{2 \times 128}$$

Cross-entropy loss, Adam (lr 0.001).

3.4.b

(4 points) Report the validation and test accuracies of your new model against the original model.

Answer.

	Val. Acc.	Test Acc.	Parameters	Runtime (s)
Original Model	97.4%	98.6%	833,794	5.7
LSTM Model	63.2%	81.1%	900,482	82.9

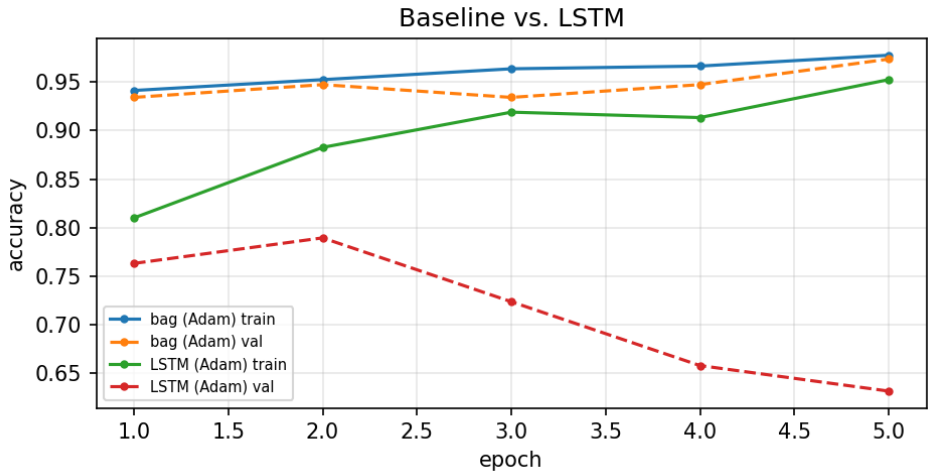


Figure 7: Baseline vs. LSTM, train/validation accuracy.

3.4.c

(2 points) Describe any differences in the train and validation accuracies between your model and the original model. In a few sentences, discuss why your model worked better or worse than the original model. Consider how input sequences are processed in an LSTM and how the presence of padding tokens might affect its learning.

Answer. The LSTM is worse than the baseline: train accuracy is 95.3% vs. 97.8%, but validation accuracy peaks at 78.9% in epoch 2 and then falls to 63.2% while validation loss rises from 0.39 to about 2.0, a clear sign of overfitting on only about 350 training articles. The final-epoch test accuracy (81.1%) is higher than the final validation accuracy; with 76 validation and 74 test articles this gap is partly noise, and I report final-epoch numbers rather than the best epoch. The LSTM reads tokens in order and can use word order, but here much of the signal seems lexical, which an average of

embeddings captures with far fewer effective degrees of freedom and a 14× shorter runtime. Padding matters for an LSTM: it carries state through every position, so hundreds of PAD steps would dilute the signal and, if only the last timestep were used, overwrite the useful state. My implementation packs sequences so padded steps are skipped and max-pools over real timesteps only.

3.5

(2 points) Look through a few of the real/fake news article pairs. In a few sentences, speculate on how the model is able to train successfully.

Answer. Each fake article was generated from the title of its real counterpart, so topic is matched within every pair and the label cannot be predicted from topic alone. The model must use stylistic and lexical cues. Statistics on the training split:

Statistic	Real	Fake
Digit tokens per 1000 tokens	18.7	4.0
Quotation marks per 1000 tokens	16.7	6.7
Share with [bracket] placeholder	7.8%	21.2%
Mean tokens	530	436

Words most over-represented in real articles: south, per, because, 9, 16, east, 2021, november, 18, told (concrete places, dates, numbers, attributions). Words most over-represented in fake articles: reminder, reflects, resilience, remarkable, unity, anticipation, recognizing, encouraging, unfolds, optimism (generic, upbeat LLM vocabulary). A bag-of-words model can learn these word-level cues without checking any fact, so high accuracy shows it separates this generator's output from these real articles, not that it detects misinformation in general. Train/val/test were split by pairs and the vocabulary comes from training text only, so validation/test scores are honest estimates of that narrower task.

4 Code Submission

(0 points) Did you submit the private GitHub repository link and ensure that the repository is accessible to the Teaching Assistant? Provide the GitHub repository link and the Google Drive model checkpoint link.

Answer. Yes.
GitHub repository: https://github.com/Umair-Ali-804/LLM_Generative_AI_Assignment01
Google Drive model checkpoints: https://drive.google.com/drive/folders/1PsVnznjIOs5PLRfHBxuf5OWr_HffjEzt

5 Collaboration Questions

5.1

(1 point) Did you collaborate with anyone on this assignment? If so, list their name or roll number and which problems you worked together on.

Answer. No. I completed this assignment on my own and did not collaborate with anyone.

5.2

(1 point) Did you find or come across code that implements any part of this assignment? If so, include full details.

Answer. No. I did not use or copy code from any other student or from any online repository that implements this assignment. I used ChatGPT for assistance; the conversation link is given at the top of this report.

AI tools used: ChatGPT, conversation link: https://chatgpt.com/c/6ad4cf9e-9d38-83e8-8605-35908d17b685